Take-Home 1

For Class 1: Big-O and Python memory

Due at the start of Class 1, on paper. Posted before the first class. About 2 hours.

The pattern: Complexity analysis. Big-O describes how the number of steps (time) and the amount of extra memory (space) grow as the input size n grows. Keep the dominant term and drop constants. Python adds a memory fact you must know: every int is an object of about 28 bytes, and a list stores an 8-byte pointer to each element, so a list of n large ints costs about 36n bytes.

Builds on earlier classes. This is the first class. Everything after it is judged with these tools.

Read

- §2.2 What Is Algorithm Analysis?
- §2.3 Big O Notation
- §2.5 Performance of Python Data Structures
- Python docs: sys.getsizeof
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
1672	Richest Customer Wealth https://leetcode.com/problems/richest-customer-wealth/	Easy
1929	Concatenation of Array https://leetcode.com/problems/concatenation-of-array/	Easy
1295	Find Numbers with Even Number of Digits https://leetcode.com/problems/find-numbers-with-even-number-of-digits/	Easy

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. For one practice problem, count the loop iterations as a function of n and write the Big-O.
8. Predict the memory of a list of 100,000 ints using 8 bytes per pointer plus 28 bytes per int. Then check your prediction with sys.getsizeof (out of class).

Bring your written plan. In class, devices are locked away.

Take-Home 2

For Class 2: Arrays and in-place updates

Due at the start of Class 2, on paper. Handed out at the end of Class 1. About 2 hours.

The pattern: Arrays: read and write indexes. A Python list is a dynamic array: indexing is O(1), appending is amortized O(1), and inserting or deleting at the front is O(n) because every later element shifts. Many array problems are solved in place with two indexes: a read index that scans and a write index that marks where the next kept value goes.

Builds on earlier classes. Class 1 gave you Big-O and the memory model. Now you apply them to the most basic structure.

Read

- §2.6 Lists
- §1.8 Getting Started with Data
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
26	Remove Duplicates from Sorted Array https://leetcode.com/problems/remove-duplicates-from-sorted-array/	Easy
27	Remove Element https://leetcode.com/problems/remove-element/	Easy
66	Plus One https://leetcode.com/problems/plus-one/	Easy

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. Which list operations in your plan are O(1) and which are O(n)? Name each one.
8. Can your plan work with O(1) extra space? What would you have to modify in the input to do it?

Bring your written plan. In class, devices are locked away.

Take-Home 3

For Class 3: Strings and prefix sums

Due at the start of Class 3, on paper. Handed out at the end of Class 2. About 2 hours.

The pattern: Strings and prefix sums. Strings are immutable sequences, so every slice or concatenation makes a new string and costs O(length). A prefix sum is a running total stored once, so any range sum becomes one subtraction. Preprocessing once to make many queries cheap is a core trade of time for space.

Builds on earlier classes. Class 2 used indexes on lists. Strings behave like read-only lists, and a prefix array is a list you build once and reuse.

Read

- §2.4 An Anagram Detection Example
- §1.8 Getting Started with Data
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
724	Find Pivot Index https://leetcode.com/problems/find-pivot-index/	Easy
28	Find the Index of the First Occurrence in a String https://leetcode.com/problems/find-the-index-of-the-first-occurrence-in-a-string/	Easy
58	Length of Last Word https://leetcode.com/problems/length-of-last-word/	Easy

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. Which operation in your plan copies a string or list? What does that copy cost?
8. If your solution will be asked many questions, what could you compute once up front?

Bring your written plan. In class, devices are locked away.

Take-Home 4

For Class 4: Hash sets and hash maps

Due at the start of Class 4, on paper. Handed out at the end of Class 3. About 2 hours.

The pattern: Hashing: sets and dicts. A hash table finds, inserts and deletes a key in O(1) on average by turning the key into a position. A set remembers what you have seen; a dict remembers something about what you have seen. Trading O(n) memory for a fast lookup turns many O(n^2) searches into O(n).

Builds on earlier classes. Class 3 stored precomputed values in a list. Now you store them by key, so lookup does not need a position.

Read

- §5.5 Hashing
- §2.7 Dictionaries
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
383	Ransom Note https://leetcode.com/problems/ransom-note/	Easy
387	First Unique Character in a String https://leetcode.com/problems/first-unique-character-in-a-string/	Easy
219	Contains Duplicate II https://leetcode.com/problems/contains-duplicate-ii/	Easy

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. What is the key and what is the value in your dict (or what does your set hold)?
8. What is the brute-force Big-O, and what does the hash table change?

Bring your written plan. In class, devices are locked away.

Take-Home 5

For Class 5: Two pointers

Due at the start of Class 5, on paper. Handed out at the end of Class 4. About 2 hours.

The pattern: Two pointers. Keep two indexes and move them according to a rule. Opposite-end pointers start at both ends of a sequence and move inward; same-direction pointers (read and write) both move forward. Each pointer only moves one way, so the whole scan is O(n) with O(1) extra space.

Builds on earlier classes. Class 2 used a read and a write index. Class 5 adds the opposite-ends version, which exploits sorted order or symmetry.

Read

- §3.18 Palindrome Checker
- §5.3 The Sequential Search
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
344	Reverse String https://leetcode.com/problems/reverse-string/	Easy
167	Two Sum II - Input Array Is Sorted https://leetcode.com/problems/two-sum-ii-input-array-is-sorted/	Medium
11	Container With Most Water https://leetcode.com/problems/container-with-most-water/	Medium

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. Where does each pointer start, and what rule decides which pointer moves?
8. Why does the rule never skip a correct answer? Write one sentence of justification.

Bring your written plan. In class, devices are locked away.

Take-Home 6

For Class 6: Sliding window

Due at the start of Class 6, on paper. Handed out at the end of Class 5. About 2 hours.

The pattern: Sliding window. A window is a contiguous block of the array. Instead of recomputing the window from scratch each time it moves, add the element that enters and subtract the element that leaves. A fixed-size window turns an O(nk) scan into O(n).

Builds on earlier classes. Class 5 moved two pointers over the array. A window is two pointers that bound a stretch, and class 3 prefix sums gave you running totals.

Read

- §2.4 An Anagram Detection Example
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html
  Sliding window has no section in Runestone. Read the section above for counting, then use the Python docs and your AI tool to study fixed and variable windows.

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
1984	Minimum Difference Between Highest and Lowest of K Scores https://leetcode.com/problems/minimum-difference-between-highest-and-lowest-of-k-scores/	Easy
1456	Maximum Number of Vowels in a Substring of Given Length https://leetcode.com/problems/maximum-number-of-vowels-in-a-substring-of-given-length/	Medium
3	Longest Substring Without Repeating Characters https://leetcode.com/problems/longest-substring-without-repeating-characters/	Medium

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. What is the state you keep for the window (a sum, a count, a set)?
8. Write the two update steps: what happens when an element enters, and when one leaves?

Bring your written plan. In class, devices are locked away.

Take-Home 7

For Class 7: Stacks

Due at the start of Class 7, on paper. Handed out at the end of Class 6. About 2 hours.

The pattern: Stack (last in, first out). A stack gives O(1) push and pop at one end. Use one when the most recent unfinished thing must be handled first: matching brackets, undo, backspaces, nested structure. A Python list is a stack with append and pop.

Builds on earlier classes. Class 4 used extra memory to remember the past. A stack remembers the past in order, and the top is always the newest item.

Read

- §3.3 What Is a Stack?
- §3.5 Implementing a Stack in Python
- §3.6 Simple Balanced Parentheses
- §3.7 Balanced Symbols (A General Case)
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
682	Baseball Game https://leetcode.com/problems/baseball-game/	Easy
155	Min Stack https://leetcode.com/problems/min-stack/	Medium
150	Evaluate Reverse Polish Notation https://leetcode.com/problems/evaluate-reverse-polish-notation/	Medium

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. What do you push, and what event causes a pop?
8. What does the stack contain at the end if the input is valid? What if it is not?

Bring your written plan. In class, devices are locked away.

Take-Home 8

For Class 8: Queues and deques

Due at the start of Class 8, on paper. Handed out at the end of Class 7. About 2 hours.

The pattern: Queue and deque (first in, first out). A queue serves items in arrival order: add at the back, remove from the front. A Python list is a poor queue because pop(0) is O(n); collections.deque gives O(1) at both ends. Queues also model streams where only recent items matter.

Builds on earlier classes. Class 7 stacks reverse order; queues keep order. Together they are the two basic linear structures.

Read

- §3.10 What Is a Queue?
- §3.12 Implementing a Queue in Python
- §3.13 Queue Simulation: Hot Potato
- §3.15 What Is a Deque?

  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
225	Implement Stack using Queues https://leetcode.com/problems/implement-stack-using-queues/	Easy
622	Design Circular Queue https://leetcode.com/problems/design-circular-queue/	Medium
1700	Number of Students Unable to Eat Lunch https://leetcode.com/problems/number-of-students-unable-to-eat-lunch/	Easy

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. Which end do items enter and which end do they leave?
8. How can you build one structure out of the other, and what does that cost per operation?

Bring your written plan. In class, devices are locked away.

Take-Home 9

For Class 9: Monotonic stack and deque

Due at the start of Class 9, on paper. Handed out at the end of Class 8. About 2 hours.

The pattern: Monotonic stack / monotonic deque. A monotonic stack keeps its contents in sorted order by popping every element that can no longer be the answer before pushing the new one. Each element is pushed once and popped once, so a scan that looks like O(n^2) ("find the next bigger value") becomes O(n). A monotonic deque does the same at both ends for sliding-window maximum.

Builds on earlier classes. Class 7 gave you the stack and class 8 the deque. Now you keep them ordered while you scan, which solves "next greater" questions.

Read

- §3.15 What Is a Deque?
- §3.17 Implementing a Deque in Python
- Python docs: collections.deque

  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

  Monotonic stacks are not in Runestone. Ask your AI tool to trace one by hand on a small array, then redo the trace yourself.

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
503	Next Greater Element II https://leetcode.com/problems/next-greater-element-ii/	Medium
901	Online Stock Span https://leetcode.com/problems/online-stock-span/	Medium
239	Sliding Window Maximum https://leetcode.com/problems/sliding-window-maximum/	Hard

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. Is the stack kept increasing or decreasing from bottom to top, and why?
8. Each element is pushed once and popped at most once. Use that fact to argue the total time.

Bring your written plan. In class, devices are locked away.

Take-Home 10

For Class 10: Linked lists

Due at the start of Class 10, on paper. Handed out at the end of Class 9. About 2 hours.

The pattern: Linked lists: pointer rewiring. A linked list stores each value in a node that points to the next node. Insert and delete at a known node are O(1), but reaching the kth node costs O(k). Most problems are solved by walking with one or two pointers and rewiring next pointers carefully, often with a dummy node in front to avoid special cases.

Builds on earlier classes. Class 5 two pointers now walk nodes instead of indexes. Class 8 queues can be built from linked nodes.

Read

- §3.19 Lists
- §3.21 Implementing an Unordered List: Linked Lists
- §3.23 Implementing an Ordered List (with analysis)
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
21	Merge Two Sorted Lists https://leetcode.com/problems/merge-two-sorted-lists/	Easy
141	Linked List Cycle https://leetcode.com/problems/linked-list-cycle/	Easy
876	Middle of the Linked List https://leetcode.com/problems/middle-of-the-linked-list/	Easy

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. Draw the list as boxes and arrows before and after your change. Which pointers change, and in what order?
8. What happens when the list is empty, has one node, or you must change the head?

Bring your written plan. In class, devices are locked away.

Take-Home 11

For Class 11: Recursion

Due at the start of Class 11, on paper. Handed out at the end of Class 10. About 2 hours.

The pattern: Recursion and divide and conquer. A recursive function solves a problem by solving smaller copies of the same problem, with a base case that stops the recursion. The time depends on how many calls you make and how much work each does; the space depends on the depth of the call stack. Repeated subproblems (like naive Fibonacci) make recursion exponential unless you remember answers.

Builds on earlier classes. Class 10 walked a list step by step; recursion lets a function call itself on the rest of the structure. The call stack is a stack from class 7.

Read

- §4.2 What Is Recursion?
- §4.4 The Three Laws of Recursion
- §4.6 Stack Frames: Implementing Recursion
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
231	Power of Two https://leetcode.com/problems/power-of-two/	Easy
22	Generate Parentheses https://leetcode.com/problems/generate-parentheses/	Medium
24	Swap Nodes in Pairs https://leetcode.com/problems/swap-nodes-in-pairs/	Medium

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. State the base case and the recursive case in one sentence each.
8. Draw the recursion tree for a small input. How many calls, and how deep does it go?

Bring your written plan. In class, devices are locked away.

Take-Home 12

For Class 12: Bit manipulation

Due at the start of Class 12, on paper. Handed out at the end of Class 11. About 2 hours.

The pattern: Bit manipulation. Integers are stored in binary, and the operators &, |, ^, <<, >> work on each bit. XOR has the useful rule that x ^ x = 0 and x ^ 0 = x, so pairs cancel. The expression x & -x isolates the lowest set bit. Bit tricks often give O(1) extra space where a hash table would use O(n).

Builds on earlier classes. Class 11 recursion on numbers halves them; bit shifts do the same in one step. Class 4 hashing used O(n) memory that XOR sometimes avoids.

Read

- §3.8 Converting Decimal Numbers to Binary Numbers
- Python docs: bitwise operations on integers
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html
  Bit manipulation has no dedicated Runestone section. Use the Python docs, then trace each operator on 8-bit numbers by hand.

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
191	Number of 1 Bits https://leetcode.com/problems/number-of-1-bits/	Easy
338	Counting Bits https://leetcode.com/problems/counting-bits/	Easy
268	Missing Number https://leetcode.com/problems/missing-number/	Easy

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. Write the numbers in your example in binary. What does each operator do to them?
8. What property (like x ^ x = 0) does your plan rely on?

Bring your written plan. In class, devices are locked away.

Take-Home 13

For Class 13: Sorting

Due at the start of Class 13, on paper. Handed out at the end of Class 12. About 2 hours.

The pattern: Sorting (merge sort and friends). Comparison sorts cannot beat O(n log n). Merge sort splits the list in half, sorts each half, and merges two sorted lists in O(n); it is stable and uses O(n) extra space. Sorting first often makes the rest of a problem easy: duplicates sit together, pairs can be found with two pointers, and binary search becomes possible.

Builds on earlier classes. Class 11 recursion gives merge sort its shape, and class 5 two pointers perform the merge step.

Read

- §5.6 Sorting
- §5.9 The Insertion Sort
- §5.11 The Merge Sort
- §5.12 The Quicksort

  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
75	Sort Colors https://leetcode.com/problems/sort-colors/	Medium
148	Sort List https://leetcode.com/problems/sort-list/	Medium
506	Relative Ranks https://leetcode.com/problems/relative-ranks/	Easy

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. Which sort will you use, and what are its time and extra space?
8. Does the order of equal elements matter in your problem (stability)?

Bring your written plan. In class, devices are locked away.

Take-Home 14

For Class 14: Binary search

Due at the start of Class 14, on paper. Handed out at the end of Class 13. About 2 hours.

The pattern: Binary search. On a sorted sequence, compare to the middle and throw away half. That gives O(log n) time and O(1) space. The hard part is the details: which half to keep, whether mid is included, and when to stop. Practice writing the loop with an invariant ("the answer, if it exists, is inside [lo, hi]").

Builds on earlier classes. Class 13 produced sorted data, which is the precondition for binary search. Class 11 halving recursion becomes a loop.

Read

- §5.4 The Binary Search
- Python docs: bisect
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
35	Search Insert Position https://leetcode.com/problems/search-insert-position/	Easy
69	Sqrt(x) https://leetcode.com/problems/sqrtx/	Easy
33	Search in Rotated Sorted Array https://leetcode.com/problems/search-in-rotated-sorted-array/	Medium

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. Write your invariant in words: what is true about [lo, hi] at the top of every loop?
8. Trace a 1-element and a 2-element array through your loop to check for off-by-one errors.

Bring your written plan. In class, devices are locked away.

Take-Home 15

For Class 15: Trees: traversal

Due at the start of Class 15, on paper. Handed out at the end of Class 14. About 2 hours.

The pattern: Binary trees: DFS and BFS. A binary tree node has a value and up to two children. Depth-first search (preorder, inorder, postorder) is recursion on the left and right subtrees; breadth-first search visits level by level with a queue. Both visit every node once, so time is O(n); space is O(height) for DFS and O(width) for BFS.

Builds on earlier classes. Class 11 recursion handles the self-similar shape of a tree, and class 8 queues drive level-order traversal.

Read

- §6.2 Examples of Trees
- §6.3 Vocabulary and Definitions
- §6.6 Nodes and References
- §6.8 Tree Traversals
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
226	Invert Binary Tree https://leetcode.com/problems/invert-binary-tree/	Easy
111	Minimum Depth of Binary Tree https://leetcode.com/problems/minimum-depth-of-binary-tree/	Easy
199	Binary Tree Right Side View https://leetcode.com/problems/binary-tree-right-side-view/	Medium

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. What does your function return for an empty tree, and for a leaf?
8. Is a DFS or a BFS the better fit, and what is the extra space of each on a skewed tree?

Bring your written plan. In class, devices are locked away.

Take-Home 16

For Class 16: Binary search trees

Due at the start of Class 16, on paper. Handed out at the end of Class 15. About 2 hours.

The pattern: Binary search tree property. In a binary search tree every value in the left subtree is smaller and every value in the right subtree is larger. Search, insert and delete follow one root-to-leaf path: O(h), which is O(log n) when balanced and O(n) when the tree is a chain. An inorder traversal of a BST visits values in sorted order.

Builds on earlier classes. Class 15 traversals visit the tree; class 14 binary search is the same halving idea, now stored in pointers.

Read

- §6.12 Binary Search Trees
- §6.13 Search Tree Operations
- §6.14 Search Tree Implementation
- §6.15 Search Tree Analysis
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
108	Convert Sorted Array to Binary Search Tree https://leetcode.com/problems/convert-sorted-array-to-binary-search-tree/	Easy
230	Kth Smallest Element in a BST https://leetcode.com/problems/kth-smallest-element-in-a-bst/	Medium
235	Lowest Common Ancestor of a Binary Search Tree https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree/	Medium

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. Use the BST property to say which subtree to visit, or to skip.
8. What does an inorder traversal of a valid BST look like, and how can you use it?

Bring your written plan. In class, devices are locked away.

Take-Home 17

For Class 17: Heaps and priority queues

Due at the start of Class 17, on paper. Handed out at the end of Class 16. About 2 hours.

The pattern: Heap / priority queue. A binary heap keeps the smallest (or largest) element at the root of an almost-complete tree stored in an array. Push and pop cost O(log n); peeking at the minimum is O(1). Use a heap when you repeatedly need "the current best": top k, kth largest, merging sorted streams. Python gives you a min-heap in heapq.

Builds on earlier classes. Class 16 trees keep order by pointers; a heap keeps partial order in a list. Class 13 sorting costs O(n log n), but a heap of size k costs O(n log k).

Read

- §6.9 Priority Queues with Binary Heaps
- §6.10 Binary Heap Operations
- §6.11 Binary Heap Implementation
- Python docs: heapq
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
1046	Last Stone Weight https://leetcode.com/problems/last-stone-weight/	Easy
973	K Closest Points to Origin https://leetcode.com/problems/k-closest-points-to-origin/	Medium
215	Kth Largest Element in an Array https://leetcode.com/problems/kth-largest-element-in-an-array/	Medium

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. Min-heap or max-heap? Python has only a min-heap: how will you handle a max-heap (hint: negate)?
8. What size does the heap keep, and why does that size give the time you claim?

Bring your written plan. In class, devices are locked away.

Take-Home 18

For Class 18: Graphs and depth-first search

Due at the start of Class 18, on paper. Handed out at the end of Class 17. About 2 hours.

The pattern: Graphs: DFS and grids. A graph is vertices and edges. Store it as an adjacency list (a dict from vertex to its neighbors), or treat a grid as an implicit graph where each cell has up to four neighbors. Depth-first search explores one path to its end before backing up, and a visited set keeps you from looping. Time is O(V + E); space is O(V).

Builds on earlier classes. Class 15 tree DFS is graph DFS on a graph with no cycles; the visited set from class 4 breaks cycles.

Read

- §7.2 Vocabulary and Definitions
- §7.5 An Adjacency List
- §7.15 General Depth-First Search
- §7.16 Depth-First Search Analysis

  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
463	Island Perimeter https://leetcode.com/problems/island-perimeter/	Easy
695	Max Area of Island https://leetcode.com/problems/max-area-of-island/	Medium
133	Clone Graph https://leetcode.com/problems/clone-graph/	Medium

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. What are the vertices, what are the edges, and how do you list the neighbors of one vertex?
8. Where do you mark a cell visited, and what goes wrong if you mark it too late?

Bring your written plan. In class, devices are locked away.

Take-Home 19

For Class 19: Breadth-first search

Due at the start of Class 19, on paper. Handed out at the end of Class 18. About 2 hours.

The pattern: Graphs: BFS and shortest path by edges. Breadth-first search explores all vertices one edge away, then two edges away, and so on, using a queue. The first time BFS reaches a vertex it has found the shortest path measured in number of edges. Multi-source BFS starts with several vertices in the queue at once. Time is O(V + E); space is O(V).

Builds on earlier classes. Class 18 DFS used a stack of calls; BFS replaces it with the class 8 queue and gains the shortest-path property.

Read

- §7.7 The Word Ladder Problem
- §7.9 Implementing Breadth-First Search
- §7.10 Breadth-First Search Analysis
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
542	01 Matrix https://leetcode.com/problems/01-matrix/	Medium
1091	Shortest Path in Binary Matrix https://leetcode.com/problems/shortest-path-in-binary-matrix/	Medium
752	Open the Lock https://leetcode.com/problems/open-the-lock/	Medium

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. What counts as one step, and what is the starting state (or states)?
8. When do you mark a state visited: when you enqueue it or when you dequeue it? Why does it matter?

Bring your written plan. In class, devices are locked away.

Take-Home 20

For Class 20: Topological sort

Due at the start of Class 20, on paper. Handed out at the end of Class 19. About 2 hours.

The pattern: Topological sort: dependencies. A topological order lists the vertices of a directed graph so that every edge goes from an earlier vertex to a later one. It exists only if there is no cycle. Kahn's algorithm repeatedly removes vertices with in-degree 0 using a queue; a DFS version orders vertices by finishing time. Both run in O(V + E) and also detect cycles.

Builds on earlier classes. Class 19 BFS with a queue becomes Kahn's algorithm. Class 18 graphs now have directions.

Read

- §7.17 Topological Sorting
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
1557	Minimum Number of Vertices to Reach All Nodes https://leetcode.com/problems/minimum-number-of-vertices-to-reach-all-nodes/	Medium
802	Find Eventual Safe States https://leetcode.com/problems/find-eventual-safe-states/	Medium
2115	Find All Possible Recipes from Given Supplies https://leetcode.com/problems/find-all-possible-recipes-from-given-supplies/	Medium

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. Which direction does an edge point: from prerequisite to course or the other way? Write the rule for in-degree.
8. How will your algorithm notice that a cycle exists?

Bring your written plan. In class, devices are locked away.

Take-Home 21

For Class 21: Shortest paths: Dijkstra

Due at the start of Class 21, on paper. Handed out at the end of Class 20. About 2 hours.

The pattern: Dijkstra with a heap. Dijkstra finds the cheapest path from a source in a graph with non-negative edge weights. Keep a min-heap of (distance, vertex); pop the closest unfinished vertex and try to improve its neighbors. With a binary heap the time is O((V + E) log V) and the space is O(V + E). Many problems hide Dijkstra by defining "distance" as something other than a sum (such as the largest step on the path).

Builds on earlier classes. Class 19 BFS is Dijkstra when every edge costs 1; the heap from class 17 replaces the plain queue.

Read

- §7.19 Shortest Path Problems
- §7.20 Dijkstra's Algorithm
- §7.21 Analysis of Dijkstra's Algorithm
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
787	Cheapest Flights Within K Stops https://leetcode.com/problems/cheapest-flights-within-k-stops/	Medium
1514	Path with Maximum Probability https://leetcode.com/problems/path-with-maximum-probability/	Medium
778	Swim in Rising Water https://leetcode.com/problems/swim-in-rising-water/	Hard

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. What is the "distance" you minimize, and how do you combine the old distance with an edge?
8. Why can a vertex be popped from the heap more than once, and how do you skip stale entries?

Bring your written plan. In class, devices are locked away.

Take-Home 22

For Class 22: Union-find

Due at the start of Class 22, on paper. Handed out at the end of Class 21. About 2 hours.

The pattern: Union-find (disjoint sets). Union-find keeps a collection of groups and supports two operations: find (which group is this item in?) and union (merge two groups). Each group is a tree whose root is its label. With path compression and union by size, both operations take nearly O(1) amortized (inverse Ackermann). Use it for connected components and for detecting the edge that closes a cycle.

Builds on earlier classes. Class 18 found components with DFS; union-find maintains them as edges arrive. It uses a parent list like the array heap of class 17.

Read

- §7.22 Prim's Spanning Tree Algorithm (context)
- VisuAlgo: Union-Find Disjoint Sets
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html
  Runestone has no union-find section. Use the VisuAlgo animation, then ask your AI tool to explain path compression and union by size.

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
990	Satisfiability of Equality Equations https://leetcode.com/problems/satisfiability-of-equality-equations/	Medium
1319	Number of Operations to Make Network Connected https://leetcode.com/problems/number-of-operations-to-make-network-connected/	Medium
721	Accounts Merge https://leetcode.com/problems/accounts-merge/	Medium

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. What are the items, and what does "same group" mean in this problem?
8. Write find with path compression in four lines, and say what changes without it.

Bring your written plan. In class, devices are locked away.

Take-Home 23

For Class 23: Tries

Due at the start of Class 23, on paper. Handed out at the end of Class 22. About 2 hours.

The pattern: Trie (prefix tree). A trie stores strings character by character along the edges of a tree, so every string that shares a prefix shares a path. Insert, search and prefix check each cost O(L) for a word of length L, independent of how many words are stored. Combine a trie with DFS to explore many words at once.

Builds on earlier classes. Class 15 trees, class 4 dicts (children stored in a dict) and class 18 DFS combine here.

Read

- §6.3 Vocabulary and Definitions
- §6.6 Nodes and References
- §2.7 Dictionaries
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html
  Runestone has no trie section. Read the sections above for tree vocabulary, then ask your AI tool to explain tries and trace an insert yourself.

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
720	Longest Word in Dictionary https://leetcode.com/problems/longest-word-in-dictionary/	Medium
648	Replace Words https://leetcode.com/problems/replace-words/	Medium
1268	Search Suggestions System https://leetcode.com/problems/search-suggestions-system/	Medium

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. What does each node store (children, end-of-word flag), and how do you represent children?
8. How many nodes can a trie of m words with at most L letters have, in the worst case?

Bring your written plan. In class, devices are locked away.

Take-Home 24

For Class 24: Backtracking

Due at the start of Class 24, on paper. Handed out at the end of Class 23. About 2 hours.

The pattern: Backtracking: choose, explore, undo. Backtracking builds an answer one choice at a time. At each step you choose an option, explore all ways to finish from there, and then undo the choice (backtrack). Prune a branch the moment it cannot lead to a valid answer. The time is the number of nodes in the search tree, usually exponential; the space is the depth of the recursion plus the current partial answer.

Builds on earlier classes. Class 11 recursion supplies the shape, class 18 DFS the idea of exploring and un-marking, and class 23 shows pruning a search by prefix.

Read

- §4.9 Complex Recursive Problems
- §4.11 Exploring a Maze
- §7.11 The Knight's Tour Problem
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
46	Permutations https://leetcode.com/problems/permutations/	Medium
39	Combination Sum https://leetcode.com/problems/combination-sum/	Medium
79	Word Search https://leetcode.com/problems/word-search/	Medium

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. What is one choice, what is the partial answer, and when is the answer complete?
8. Which branches can you cut early? State the pruning rule.

Bring your written plan. In class, devices are locked away.

Take-Home 25

For Class 25: Greedy algorithms

Due at the start of Class 25, on paper. Handed out at the end of Class 24. About 2 hours.

The pattern: Greedy choice. A greedy algorithm makes the locally best choice at every step and never reconsiders. It is correct only when you can argue that the local choice cannot hurt the final answer (an exchange argument), and it is usually fast: a sort or a single scan, O(n log n) or O(n). Always test your rule against a small counterexample before trusting it.

Builds on earlier classes. Class 13 sorting often sets up a greedy scan. Class 24 explored every choice; greedy commits to one.

Read

- §7.22 Prim's Spanning Tree Algorithm (a greedy algorithm)
- §4.12 Dynamic Programming (greedy vs. DP on coin change)
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html
  Runestone has no greedy chapter. Use Erickson, Algorithms (free online) for exchange arguments, or ask your AI tool for one.

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
455	Assign Cookies https://leetcode.com/problems/assign-cookies/	Easy
122	Best Time to Buy and Sell Stock II https://leetcode.com/problems/best-time-to-buy-and-sell-stock-ii/	Medium
134	Gas Station https://leetcode.com/problems/gas-station/	Medium

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. What is your greedy rule in one sentence?
8. Try to break it: write a small input where a different choice would do better. If you cannot, say why.

Bring your written plan. In class, devices are locked away.

Take-Home 26

For Class 26: Intervals and sweep line

Due at the start of Class 26, on paper. Handed out at the end of Class 25. About 2 hours.

The pattern: Intervals: sort, then sweep. Most interval problems start by sorting by start time (or end time). A single left-to-right sweep then merges overlaps, counts how many intervals are active at once, or removes the minimum number to avoid overlap. A heap can track the active intervals when the sweep needs "the one that ends first" or "the tallest right now".

Builds on earlier classes. Class 13 sorting and class 25 greedy scans combine; class 17 heaps handle the harder sweep problems.

Read

- §5.11 The Merge Sort
- §6.9 Priority Queues with Binary Heaps
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html
  Intervals are not a Runestone section. Re-read the sorting and heap sections above and ask your AI tool to explain a sweep line.

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
228	Summary Ranges https://leetcode.com/problems/summary-ranges/	Easy
57	Insert Interval https://leetcode.com/problems/insert-interval/	Medium
452	Minimum Number of Arrows to Burst Balloons https://leetcode.com/problems/minimum-number-of-arrows-to-burst-balloons/	Medium

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. What do you sort by, and what does that guarantee for the scan?
8. Write the overlap test for two intervals (a, b) and (c, d) after sorting.

Bring your written plan. In class, devices are locked away.

Take-Home 27

For Class 27: Dynamic programming: 1D

Due at the start of Class 27, on paper. Handed out at the end of Class 26. About 2 hours.

The pattern: Dynamic programming: one dimension. Dynamic programming solves a problem by combining answers to smaller versions of the same problem, and stores each answer so it is computed once. Write the answer for state i in terms of earlier states (the recurrence), pick the base cases, and fill the table in order. A 1D table gives O(n) time and O(n) space, often reducible to O(1) by keeping only the last values.

Builds on earlier classes. Class 11 naive recursion repeated work; DP is that recursion with memory (class 4 dict) or filled bottom-up in a list (class 2).

Read

- §4.12 Dynamic Programming
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
70	Climbing Stairs https://leetcode.com/problems/climbing-stairs/	Easy
746	Min Cost Climbing Stairs https://leetcode.com/problems/min-cost-climbing-stairs/	Easy
213	House Robber II https://leetcode.com/problems/house-robber-ii/	Medium

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. Define the state in words: "dp[i] means ...". Write the recurrence and the base cases.
8. Which earlier states does dp[i] need? Can you keep only those?

Bring your written plan. In class, devices are locked away.

Take-Home 28

For Class 28: Dynamic programming: sequences

Due at the start of Class 28, on paper. Handed out at the end of Class 27. About 2 hours.

The pattern: Dynamic programming: sequences. When the state is a position in a sequence, the recurrence often asks "what is the best answer that ends at i?" or "what is the best answer using the first i items?". Longest increasing subsequence and longest common subsequence are the standard examples. Time is the number of states times the work per state: O(n^2) for LIS with a simple scan, O(mn) for LCS.

Builds on earlier classes. Class 27 introduced 1D tables; here a state depends on all earlier states, and class 29 will use two sequences at once.

Read

- §4.12 Dynamic Programming (read again)
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html
  Runestone covers the idea but not these problems. Use Erickson, Algorithms (free online), Dynamic Programming chapter, and your AI tool.

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
139	Word Break https://leetcode.com/problems/word-break/	Medium
53	Maximum Subarray https://leetcode.com/problems/maximum-subarray/	Medium
152	Maximum Product Subarray https://leetcode.com/problems/maximum-product-subarray/	Medium

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. Define the state: "dp[i] means ...". Does it end at i, or use the first i items?
8. How many states are there, and how much work does each one need? Multiply to get the time.

Bring your written plan. In class, devices are locked away.

Take-Home 29

For Class 29: Dynamic programming: 2D tables

Due at the start of Class 29, on paper. Handed out at the end of Class 28. About 2 hours.

The pattern: Dynamic programming: two dimensions. When two sequences (or a grid) are involved, the state needs two indexes: dp[i][j] is the answer for the first i items of one input and the first j of the other. Each cell depends on its neighbors above and to the left. Time and space are both O(mn); often only the previous row is needed, which brings space to O(n).

Builds on earlier classes. Class 28 used one sequence; now two sequences or a grid give a table. Class 24 pruned a search; DP removes the repeated work instead.

Read

- §4.12 Dynamic Programming (read again)
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html
  Use Erickson, Algorithms (free online), Dynamic Programming chapter, for edit distance, and your AI tool for table-filling traces.

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
62	Unique Paths https://leetcode.com/problems/unique-paths/	Medium
64	Minimum Path Sum https://leetcode.com/problems/minimum-path-sum/	Medium
221	Maximal Square https://leetcode.com/problems/maximal-square/	Medium

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. Define dp[i][j] in words and write the base row and base column.
8. Draw the table for a tiny input and fill three cells by hand.

Bring your written plan. In class, devices are locked away.

Take-Home 30

For Class 30: Capstone: choosing the pattern

Due at the start of Class 30, on paper. Handed out at the end of Class 29. About 2 hours.

The pattern: Mixed review: all patterns. The final class has no new pattern. You are given two hard problems with no topic label, and you must decide which structure and algorithm to use, justify it, and compute time, space and memory. Review the one-line idea of every pattern in this course: arrays, hashing, two pointers, windows, stacks, queues, linked lists, recursion, bits, sorting, binary search, trees, heaps, graphs, union-find, tries, backtracking, greedy, intervals, DP.

Builds on earlier classes. Every earlier class. Hard problems usually combine two patterns.

Read

- §2.8 Summary (algorithm analysis)
- §5.13 Summary (searching and sorting)
- §6.20 Summary (trees)
- §7.23 Summary (graphs)
  Book: https://runestone.academy/ns/books/published/pythonds3/index.html

Practice on LeetCode (do all, in this order)

#	Problem and link	Level
42	Trapping Rain Water https://leetcode.com/problems/trapping-rain-water/	Hard
76	Minimum Window Substring https://leetcode.com/problems/minimum-window-substring/	Hard
23	Merge k Sorted Lists https://leetcode.com/problems/merge-k-sorted-lists/	Hard

Plan before code (on paper; pick one practice problem)

1. The problem in your own words, and the input size n.
2. The brute-force idea and its Big-O time.
3. The pattern you will use, and why it fits.
4. The steps in plain English (no code yet).
5. Expected time and space, and the memory for n = 100,000.
6. Two edge cases.

Also answer, for this pattern:

7. For each practice problem write the pattern name and the brute-force Big-O, with no code.
8. Write a one-page pattern cheat sheet: for each pattern, the clue in the problem text that tells you to use it.

Bring your written plan. In class, devices are locked away.